

Simulador de Trayectorias en Detector Virtual de Partículas

Elisa Inostroza*

Universidad técnica Federico Santa María.

(Dated: 17 de abril de 2026)

En física de partículas, la reconstrucción de trayectorias es una de las tareas fundamentales en experimentos modernos. Cuando una partícula cargada atraviesa un detector inmerso en un campo magnético, su trayectoria se curva debido a la fuerza de Lorentz. Midiendo puntos discretos de paso (hits) en capas del detector, es posible reconstruir el momento transversal, la dirección de la partícula y otras propiedades cinemáticas.

El objetivo de este proyecto es desarrollar un simulador computacional capaz de modelar el movimiento relativista de partículas cargadas en campos electromagnéticos y generar una geometría cilíndrica para registrar sus puntos de intersección. En esta entrega se implementaron los puntos (1) y (2) del enunciado, un integrador numérico Runge-Kutta de cuarto orden (RK4) relativista para la ecuación de movimiento y un detector cilíndrico que registra intersecciones de la trayectoria con capas radiales.

I. MODELO FÍSICO

El movimiento de una partícula cargada en presencia de campos eléctricos y magnéticos se describe mediante la ecuación relativista de Lorentz

$$\frac{d\vec{p}}{dt} = q(\vec{E} + \vec{v} \times \vec{B}) \quad (1)$$

Donde q es la carga de la partícula, $\vec{E}$ es el campo eléctrico, $\vec{B}$ es el campo magnético, $\vec{v}$ la velocidad y $\vec{p}$ el momento relativista.

En relatividad especial, el momento está relacionado con la velocidad por

$$\vec{p} = \gamma m \vec{v} \quad (2)$$

con

$$\gamma = \sqrt{1 + \frac{p^2}{m^2 c^2}} \quad (3)$$

Finalmente, la velocidad se obtiene como

$$\vec{v} = \frac{\vec{p}}{\gamma m} \quad (4)$$

II. SISTEMA DINÁMICO

El sistema físico se formula como un sistema dinámico en seis variables, posición y momento. Se define el estado

$$\text{Estado}(t) = (\vec{r}(t), \vec{p}(t)) \quad (5)$$

Donde $\vec{r}(t) = (x, y, z)$ y $\vec{p}(t) = (p_x, p_y, p_z)$

El sistema diferencial queda

$$\frac{d\vec{r}}{dt} = \vec{v} \quad (6)$$

$$\frac{d\vec{p}}{dt} = q(\vec{E} + \vec{v} \times \vec{B}) \quad (7)$$

Este sistema no tiene solución analítica simple, por lo cual es necesario utilizar integración numérica.

III. MÉTODO NUMÉRICO: RUNGE-KUTTA DE CUARTO ORDEN

Para resolver el sistema de ecuaciones diferenciales se utilizó el método de Runge-Kutta de cuarto orden. Este método aproxima la evolución del estado en un paso de tiempo dt mediante cuatro evaluaciones intermedias de la derivada

$$k_1 = f(t_n, y_n) \quad (8)$$

$$k_2 = f(t_n + \frac{dt}{2}, y_n + k_1 \frac{dt}{2}) \quad (9)$$

$$k_3 = f(t_n + \frac{dt}{2}, y_n + k_2 \frac{dt}{2}) \quad (10)$$

$$k_4 = f(t_n + dt, y_n + k_3 dt) \quad (11)$$

* Correo institucional: einostroza@usm.cl

Luego, el estado queda como

$$y_{n+1} = y_n + \frac{dt}{6}(k_1 + 2k_2 + 2k_3 + k_4) \quad (12)$$

En este caso, y corresponde al estado $(\vec{r}, \vec{p})$ y f corresponde a las ecuaciones descritas anteriormente.

IV. IMPLEMENTACIÓN COMPUTACIONAL DEL INTEGRADOR RELATIVISTA

La implementación computacional se realizó en C++, para mantener el código ordenado y fácil de extender, el proyecto se organizó en varios archivos separados. Se utilizaron archivos de cabecera (.h) para declarar estructuras y funciones, y archivos fuente (.cc) para implementar la lógica principal.

Se crearon estructuras de datos específicas para representar vectores tridimensionales (Vec), el estado dinámico (Estado) y sus derivadas (Derivada). Esto permite trabajar con vectores de forma más natural gracias a que además se definieron las operaciones suma, resta, multiplicación por un escalar y producto cruz, tratando (x,y,z) como un solo objeto. Esto hace que el código sea más claro, ordenado y fácil de leer, y además evita cálculos repetitivos componente por componente.

Se implementaron dos funciones, siendo la primera “calcular_derivada” y la segunda “rk4”

```
1 Derivada calcular_derivada(const Estado&
    estado_actual, double m, double q,
    const Vec& E, const Vec& B)
```

```
1 Estado rk4(const Estado& estado_actual,
    double dt, double m, double q, const
    Vec& E, const Vec& B)
```

Gracias a la función “calcular_derivada”, es posible evaluar en cada instante las ecuaciones de movimiento relativistas, obteniendo $\frac{d\vec{r}}{dt}$ y $\frac{d\vec{p}}{dt}$. Luego, el método numérico Runge-Kutta de cuarto orden (RK4) utiliza estas derivadas para integrar el sistema diferencial y actualizar el estado de la partícula paso a paso. Al repetir este procedimiento en muchos intervalos de tiempo, se obtiene una aproximación numérica de la trayectoria completa de la partícula dentro del detector.

V. GEOMETRÍA CILÍNDRICA TIPO CMS/ATLAS

La geometría del detector se modeló como una serie de cilindros concéntricos alrededor del eje z , cada capa se define con radio R_i .

Se sabe que la condición geométrica para una capa cilíndrica es:

$$\rho = \sqrt{x^2 + y^2} = R_i \quad (13)$$

Se define un conjunto de capas

$$R = \{R_1, R_2, \dots, R_n\} \quad (14)$$

Durante la simulación se evalúa en cada paso temporal si la partícula cruzó alguna capa comparando el radio previo ρ_{prev} y el radio actual ρ . Si

$$\rho_{prev} < R_i \quad \text{y} \quad \rho \geq R_i \quad (15)$$

entonces se considera que la partícula intersectó la capa i . Se registra la posición de cruce como un “hit”. Esto se implementó en la función “detectar_intersecciones”

```
1 vector<Vec> detectar_intersecciones(Estado
    estado, double dt, int N, double m,
    double q, const Vec& E, const Vec& B,
    const vector<double>& capas,
    vector<Estado>& trayectoria_completa);
```

Además se almacena la trayectoria completa de la partícula para posterior análisis y visualización.

VI. EXPORTACIÓN DE DATOS Y USO DE ROOT

Para facilitar el análisis y visualización, se implementó la exportación de la trayectoria completa y de los hits a un archivo ROOT usando las clases TFile y TTree, en el cual se guardan las variables:

- x, y, z: Las coordenadas espaciales de la partícula en cada paso del integrador
- es_hit: Una variable booleana que nos permite identificar si ese punto específico de la trayectoria corresponde a una intersección con una de las capas del detector.

VII. RESULTADOS PRELIMINARES

Se realizó una prueba, simulando un protón con masa $1.6726219 \times 10^{-27}$ kg y carga $1.602176634 \times 10^{-19}$ C en un campo magnético uniforme

$$\vec{B} = (0, 0, 3.8)\text{T} \quad (16)$$

y campo eléctrico nulo

$$\vec{E} = (0, 0, 0) \text{V/m} \quad (17)$$

Se utilizaron capas cilíndricas de radios

$$R = \{0.05, 0.10, 0.20, 0.40, 0.60\} \text{m} \quad (18)$$

Para el cual, la simulación registró correctamente el número de capas cruzadas y exportó tanto la trayectoria como los hits a un archivo ROOT. Se logró observar una trayectoria helicoidal como era esperado teóricamente, esto se verificó por medio de un código de Python, el cual graficó los datos de la trayectoria guardados previamente en el archivo root.

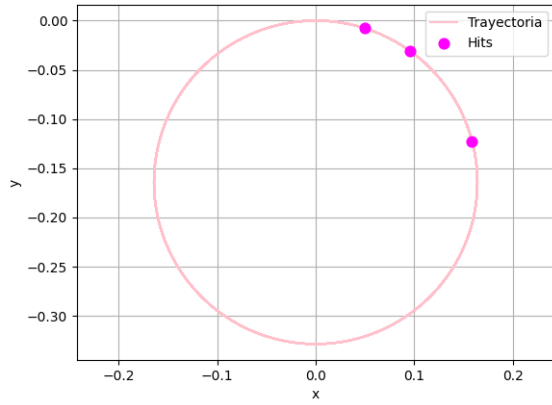


Figura 1. Proyección de la trayectoria en el eje XY